

上一章介绍的向量结构中，各数据项的物理存放位置与逻辑次序完全对应，故可通过秩直接访问对应的元素，此即所谓“循秩访问”（**call-by-rank**）。这种访问方式，如同根据具体的城市名、街道名和门牌号，直接找到某人。本章将要介绍的列表，与向量同属序列结构的范畴，其中的元素也构成一个线性逻辑次序；但与向量极为不同的是，元素的物理地址可以任意。

为保证对列表元素访问的可行性，逻辑上互为前驱和后继的元素之间，应维护某种索引关系。这种索引关系，可抽象地理解为被索引元素的位置（**position**），故列表元素是“循位置访问”（**call-by-position**）的；也可形象地理解为通往被索引元素的链接（**link**），故亦称作“循链接访问”（**call-by-link**）。这种访问方式，如同通过你的某位亲朋，找到他/她的亲朋、亲朋的亲朋、……。注意，向量中的秩同时对应于逻辑和物理次序，而位置仅对应于逻辑次序。

本章的讲解，将围绕列表结构的高效实现逐步展开，包括其ADT接口规范以及对应的算法。此外还将针对有序列表，系统地介绍排序等经典算法，并就其性能做一分析和对比。

§ 3.1 从向量到列表

不同数据结构内部的存储与组织方式各异，其操作接口的使用方式及时空性能也不尽相同。在设计或选用数据结构时，应从实际应用的需求出发，先确定功能规范及性能指标。比如，引入列表结构的目的，就在于弥补向量结构在解决某些应用问题时，在功能及性能方面的不足。二者之间的差异，表面上体现于对外的操作方式，但根源则在于其内部存储方式的不同。

3.1.1 从静态到动态

数据结构支持的操作，通常无非静态和动态两类：前者仅从中获取信息，后者则会修改数据结构的局部甚至整体。以第2章基于数组实现的向量结构为例，其**size()**和**get()**等静态操作均可在常数时间内完成，而**insert()**和**remove()**等动态操作却都可能需要线性时间。究其原因，在于“各元素物理地址连续”的约定——此即所谓的“静态存储”策略。

得益于这种策略，可在 $O(1)$ 时间内由秩确定向量元素的物理地址；但反过来，在添加（删除）元素之前（之后），又不得不移动 $O(n)$ 个后继元素。可见，尽管如此可使静态操作的效率达到极致，但就动态操作而言，局部的修改可能引起大范围甚至整个数据结构的调整。

列表（**list**）结构尽管也要求各元素在逻辑上具有线性次序，但对其物理地址却未作任何限制——此即所谓“动态存储”策略。具体地，在其生命期内，此类数据结构将随着内部数据的需要，相应地分配或回收局部的数据空间。如此，元素之间的逻辑关系得以延续，却不必与其物理次序相关。作为补偿，此类结构将通过指针或引用等机制，来确定各元素的实际物理地址。

例如，链表（**linked list**）就是一种典型的动态存储结构。其中的数据，分散为一系列称作节点（**node**）的单位，节点之间通过指针相互索引和访问。为了引入新节点或删除原有节点，只需在局部，调整少量相关节点之间的指针。这就意味着，采用动态存储策略，至少可以大大降低动态操作的成本。

3.1.2 由秩到位置

改用以上动态存储策略之后，在提高动态操作效率的同时，却又不得不舍弃原静态存储策略中循秩访问的方式，从而造成静态操作性能的下降。

以采用动态存储策略的线性结构（比如链表）为例。尽管按照逻辑次序，每个数据元素依然具有秩这一指标，但为了访问秩为 r 的元素，我们只能顺着相邻元素之间的指针，从某一端出发逐个扫描各元素，经过 r 步迭代后才能确定该元素的物理存储位置。这意味着，原先只需 $O(1)$ 时间的静态操作，此时的复杂度也将线性正比于被访问元素的秩，在最坏情况下等于元素总数 n ；即便在各元素被访问概率相等的情况下，平均而言也需要 $O(n)$ 时间。

对数据结构的访问方式，应与其存储策略相一致。此时，既然继续沿用循秩访问的方式已非上策，就应更多地习惯于通过位置，来指代并访问动态存储结构中的数据元素。与向量中秩的地位与功能类似，列表中的位置也是指代各数据元素的一个标识性指标，借助它可以便捷地（比如在常数时间内）得到元素的物理存储地址。各元素的位置，通常可表示和实现为联接于元素之间的指针或引用。因此，基于此类结构设计算法时，应更多地借助逻辑上相邻元素之间的位置索引，以实现目标元素的快速定位和访问，并进而提高算法的整体效率。

3.1.3 列表

与向量一样，列表也是由具有线性逻辑次序的一组元素构成的集合：

$$L = \{ a_0, a_1, \dots, a_{n-1} \}$$

列表是链表结构的一般化推广，其中的元素称作节点（node），分别由特定的位置或链接指代。与向量一样，在元素之间，也可定义前驱、直接前驱，以及后继、直接后继等关系；相对于任意元素，也有定义对应的前缀、后缀等子集。

§ 3.2 接口

如上所述，作为列表的基本组成单位，列表节点除需保存对应的数据项，还应记录其前驱和后继的位置，故需将这些信息及相关操作组成列表节点对象，然后参与列表的构建。

本节将给出列表节点类与列表类的接口模板类描述，稍后逐一讲解各接口的具体实现。

3.2.1 列表节点

■ ADT接口

作为一种抽象数据类型，列表节点对象应支持以下操作接口。

表3.1 列表节点ADT支持的操作接口

操 作 接 口	功 能
data()	当前节点所存数据对象
pred()	当前节点前驱节点的位置
succ()	当前节点后继节点的位置
insertAsPred(e)	插入前驱节点，存入被引用对象e，返回新节点位置
insertAsSucc(e)	插入后继节点，存入被引用对象e，返回新节点位置